

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

ÁLISON CHRISTIAN REBOUÇAS VIDAL DE CARVALHO

**ANÁLISE COMPARATIVA DE BANCOS DE DADOS RELACIONAIS E
ORIENTADOS A GRAFOS PARA SISTEMAS DE RECOMENDAÇÃO EM E-
COMMERCE**

CORNÉLIO PROCÓPIO

2026

ÁLISON CHRISTIAN REBOUÇAS VIDAL DE CARVALHO

**ANÁLISE COMPARATIVA DE BANCOS DE DADOS RELACIONAIS E
ORIENTADOS A GRAFOS PARA SISTEMAS DE RECOMENDAÇÃO EM E-
COMMERCE**

**Comparative Analysis of Relational and Graph-Oriented Databases for
Recommendation Systems in E-commerce**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção do título
de Bacharel em Engenharia de Software, da
Universidade Tecnológica Federal do Paraná
(UTFPR).
Orientador: Prof. Dr. Eduardo Cotrin Teixeira.

CORNÉLIO PROCÓPIO

2026



Esta licença permite download e compartilhamento do trabalho desde que sejam atribuídos créditos ao(s) autor(es), sem a possibilidade de alterá-lo ou utilizá-lo para fins comerciais. Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

AGRADECIMENTOS

Dedico este trabalho à memória da minha mãe, Ana Cristina Rebouças Vidal de Carvalho, que sempre torceu por mim, acreditou na minha caminhada e me ofereceu amor, apoio e incentivo ao longo da vida. Sua presença, seus ensinamentos e sua força permanecem comigo e foram fundamentais para que eu chegasse até esta etapa.

Agradeço ao meu pai, Senio Vidal de Carvalho, pelo apoio incondicional, tanto financeiro quanto psicológico, que me proporcionou a base necessária para concluir esta etapa da minha formação.

Aos meus amigos, pelo incentivo constante e pela presença nos momentos mais desafiadores.

A todos os professores do curso de Engenharia de Software, pela dedicação, compromisso e por todo o conhecimento compartilhado ao longo desta jornada.

Ao meu orientador, Prof. Dr. Eduardo Cotrin Teixeira, pela paciência, disponibilidade, compreensão e pelas valiosas orientações que foram fundamentais para o desenvolvimento deste trabalho.

RESUMO

Sistemas de recomendação são mecanismos computacionais utilizados para sugerir itens de possível interesse aos usuários, com base em dados de interação, histórico de consumo, similaridade entre perfis ou características dos próprios itens. O crescimento exponencial do comércio eletrônico tem impulsionado a necessidade de sistemas de recomendação mais eficientes e escaláveis, e a manipulação dos dados envolvidos nesse processo tem importância central para a obtenção de resultados satisfatórios. Nesse contexto, este trabalho apresenta uma análise comparativa de desempenho e expressividade entre bancos de dados relacionais e bancos de dados orientados a grafos no contexto de recomendações para e-commerce. Para a análise, foi utilizado o conjunto de dados público da Olist (plataforma brasileira de comércio eletrônico que disponibiliza dados anonimizados de pedidos), modelado no PostgreSQL com uso de indexação B-Tree, e no Neo4j com Grafos de Propriedades Rotuladas, e submetido a um benchmark automatizado com amostragem dinâmica para evitar viés de memória cache. O experimento testou desde filtragens colaborativas, que são aquelas baseadas no histórico de usuários com preferências semelhantes, até algoritmos de menor caminho e testes de estresse com múltiplas junções. Os resultados demonstraram comportamentos distintos entre os SGBDs em situações específicas: o PostgreSQL foi superior em consultas de baixa profundidade, isto é, consultas que percorrem poucas relações entre entidades, resolvendo análises de cesta de compras em médias de 0,35 ms contra 2,90 ms do Neo4j. Contudo, ao submeter os motores a um estresse de rede estendida com quatro saltos lógicos, o PostgreSQL entrou em timeout por esgotamento de processamento, enquanto o Neo4j manteve execuções válidas. Na busca por caminhos mais curtos, o banco orientado a grafos foi mais de 160 vezes mais rápido. Conclui-se que, nos cenários avaliados, o modelo relacional mostrou-se mais eficiente para recomendações rasas em bases de tamanho moderado, enquanto a adoção de grafos torna-se especialmente adequada para a descoberta de redes ocultas e para a manutenção de latência estável em consultas de alta conectividade.

Palavras-chave: Banco de Dados; Neo4j; PostgreSQL; Sistemas de Recomendação; E-commerce.

ABSTRACT

Recommendation systems are computational mechanisms used to suggest items of possible interest to users based on interaction data, consumption history, profile similarity, or characteristics of the items themselves. The exponential growth of e-commerce has increased the need for more efficient and scalable recommendation systems, and the management of the data involved in this process plays a central role in obtaining satisfactory results. In this context, this work presents a comparative analysis of performance and expressiveness between relational databases and graph-oriented databases in the context of recommendations for e-commerce. For the analysis, the public Olist dataset (a Brazilian e-commerce platform that provides anonymized order data) was used, modeled in PostgreSQL with B-Tree indexing, and in Neo4j with Labeled Property Graphs, and submitted to an automated benchmark with dynamic sampling to avoid memory cache bias. The experiment tested queries ranging from collaborative filtering, which is based on the history of users with similar preferences, to shortest-path algorithms and stress tests with multiple joins. The results demonstrated distinct behaviors between the DBMSs in specific situations: PostgreSQL was superior in low-depth queries, that is, queries that traverse few relationships between entities, solving market basket analyses in averages of 0.35 ms against 2.90 ms for Neo4j. However, when the engines were submitted to an extended network stress test with four logical hops, PostgreSQL reached a timeout due to processing exhaustion, while Neo4j maintained valid executions. In the search for shortest paths, the graph-oriented database was more than 160 times faster. It is concluded that, in the evaluated scenarios, the relational model proved more efficient for shallow recommendations in moderately sized datasets, while the adoption of graphs becomes especially suitable for discovering hidden networks and maintaining stable latency in highly connected queries.

Keywords: Databases; Neo4j; PostgreSQL; Recommendation Systems; E-commerce.

LISTA DE FIGURAS

Figura 1 - Mecanismo de Junção (Join) e Busca por Índice B-Tree no Modelo Relacional	7
Figura 2 - Principais Categorias de Bancos de Dados NoSQL	8
Figura 3 - Exemplo de Representação em Grafo para Análise de Caminhos ...	9
Figura 4 - Estrutura Conceitual de um Grafo de Propriedades Rotuladas (LPG)	10
Figura 5 - Matriz de Interação e Mecanismo de Dedução por Filtragem Colaborativa.....	12
Figura 6 - Estrutura Relacional Simplificada do Dataset Olist	19
Figura 7 - Arquitetura Geral e Fluxo de Execução do Experimento	22
Figura 8 - Diagrama Entidade-Relacionamento do Modelo Relacional (PostgreSQL)	27
Figura 9 - Modelo de Grafo (Neo4j)	28

LISTA DE TABELAS

Tabela 1 - Exemplos de Estruturas Relacionais Criadas no PostgreSQL	20
Tabela 2 - Síntese das Consultas Implementadas no Benchmark	23
Tabela 3 - Parâmetros de Execução do Benchmark	29
Tabela 4 - Tempo de Resposta (ms) para Consultas de Baixa Profundidade...	32
Tabela 5 - Escalonamento de Profundidade e Desempenho (ms)	34
Tabela 6 - Tempo de Resposta (ms) para Descoberta de Caminho (Q5).....	35

LISTA DE ABREVIATURAS E SIGLAS

3FN	Terceira Forma Normal
ACID	Atomicidade, Consistência, Isolamento e Durabilidade
BDOG	Banco de Dados Orientado a Grafos
CPU	Central Processing Unit
CSV	Comma-Separated Values
DER	Diagrama Entidade-Relacionamento
ETL	Extract, Transform, Load
FBC	Filtragem Baseada em Conteúdo
FC	Filtragem Colaborativa
FK	Foreign Key
GPU	Graphics Processing Unit
I/O	Input/Output
LOC	Lines of Code
LPG	Labeled Property Graph
NoSQL	Not Only SQL
NVMe	Non-Volatile Memory Express
RAM	Random Access Memory
RF	Requisito Funcional
RNF	Requisito Não Funcional
SGBD	Sistema Gerenciador de Banco de Dados
SGBDR	Sistema Gerenciador de Banco de Dados Relacional
SGBDOR	Sistema Gerenciador de Banco de Dados Objeto-Relacional
SQL	Structured Query Language
SR	Sistema de Recomendação
SSD	Solid-State Drive
TCO	Total Cost of Ownership
VS Code	Visual Studio Code

SUMÁRIO

1	INTRODUÇÃO	1
1.1	Problema de Pesquisa	2
1.2	Objetivo	2
1.3	Justificativa.....	3
1.4	Organização do Trabalho	4
2	FUNDAMENTAÇÃO TEÓRICA	5
2.1	Banco de Dados	5
2.2	Banco de Dados Relacional	6
2.3	Banco de Dados NoSQL	7
2.4	Banco de Dados Orientado a Grafos	9
2.5	Sistemas de Recomendação	11
2.6	Filtragem Colaborativa.....	12
2.7	Filtragem Baseada em Conteúdo.....	13
2.8	Ferramentas Utilizadas	14
3	METODOLOGIA	17
3.1	Ferramentas e Tecnologias	17
3.2	Objeto de Estudo.....	18
3.3	Arquitetura do Experimento	19
4	DESENVOLVIMENTO	23
4.1	Consultas Implementadas no Experimento	23
4.2	Modelagem de Dados.....	26
4.3	Procedimento de Execução do Benchmark	29
5	ANÁLISE DOS RESULTADOS E DISCUSSÃO.....	32
5.1	Desempenho em Consultas de Baixa Profundidade.....	32
5.2	Teste de Estresse por Aumento de Profundidade.....	33
5.3	Descoberta de Caminhos e Expressividade	35
6	CONSIDERAÇÕES FINAIS	36
	REFERÊNCIAS.....	38

1 INTRODUÇÃO

A popularização da internet e o avanço das tecnologias da informação e comunicação transformaram o comportamento de consumo e ampliaram a relevância do comércio eletrônico (e-commerce). Com o aumento no volume de transações e interações digitais, plataformas de vendas online passaram a armazenar dados sobre clientes, pedidos, produtos, vendedores e preferências de consumo. Nesse contexto, os Sistemas de Recomendação (SR) tornaram-se mecanismos relevantes para sugerir itens aos usuários com base em histórico de interações, padrões de consumo e similaridade entre perfis ou produtos (RICCI; ROKACH; SHAPIRA, 2011; AGGARWAL, 2016).

Tradicionalmente, a persistência e o processamento dos dados que alimentam esses sistemas têm sido realizados por Sistemas Gerenciadores de Bancos de Dados Relacionais (SGBDR), como o PostgreSQL. Baseados no modelo relacional proposto por Edgar F. Codd na década de 1970, esses bancos organizam os dados em tabelas relacionadas e oferecem mecanismos de integridade estrutural e consistência transacional (CODD, 1970; ELMASRI; NAVATHE, 2011). No entanto, quando sistemas de recomendação passam a explorar conexões entre usuários, pedidos, categorias e produtos, o modelo relacional pode exigir múltiplas operações de junção (joins) para reconstruir essas relações durante a execução das consultas.

Em resposta a demandas que nem sempre se ajustam ao modelo relacional, tecnologias NoSQL (*Not Only SQL*) passaram a ser utilizadas em diferentes contextos de armazenamento e consulta de dados. Dentro desse conjunto, destacam-se os Bancos de Dados Orientados a Grafos (BDOG), exemplificados por tecnologias como o Neo4j. Diferentemente do modelo relacional, a abordagem baseada em grafos trata os relacionamentos como entidades de primeira classe, isto é, como elementos armazenados explicitamente no modelo e que podem possuir direção, tipo e propriedades próprias. Por meio do conceito de adjacência livre de índice (*index-free adjacency*), os relacionamentos passam a ser representados como conexões diretas entre entidades, permitindo que consultas de travessia percorram partes específicas da rede de dados em vez de reconstruir todas as associações por meio de junções tabulares (ROBINSON; WEBBER; EIFREM, 2015).

1.1 Problema de Pesquisa

O projeto de um sistema de recomendação envolve decisões sobre a forma de persistir, consultar e relacionar os dados utilizados no processo de recomendação. Para a manipulação desses dados, podem ser utilizadas tecnologias relacionais, maduras e amplamente difundidas, ou bancos orientados a grafos, que representam entidades e conexões de forma explícita. Embora ambas as abordagens possam ser aplicadas ao mesmo domínio, elas diferem na forma de modelagem, na linguagem de consulta e nos mecanismos utilizados para acessar os relacionamentos entre os dados.

Bancos de Dados Orientados a Grafos apresentam boa aderência a problemas baseados em conexões e travessias, como redes sociais, recomendação, análise de rotas e detecção de fraudes (ROBINSON; WEBBER; EIFREM, 2015). Entretanto, em cenários específicos de comércio eletrônico, ainda é possível avaliar experimentalmente como essa abordagem se comporta em comparação com um banco relacional otimizado por índices. Além do tempo de resposta das consultas, também é relevante observar a expressividade das linguagens utilizadas, entendida neste trabalho como a clareza, a concisão e a complexidade estrutural das consultas escritas em SQL e Cypher.

Diante disso, esta pesquisa buscou responder ao seguinte problema: como variam o desempenho computacional e a expressividade das consultas ao implementar a lógica de um sistema de recomendação, baseado em filtragem colaborativa e análise de perfil similar, em um banco de dados relacional (PostgreSQL) e em um banco de dados orientado a grafos (Neo4j), utilizando um conjunto de dados reais do e-commerce brasileiro?

1.2 Objetivo

Este trabalho tem como objetivo comparar o desempenho quantitativo, mensurado por meio do tempo de resposta das consultas, e a expressividade das consultas, entendida como a concisão, a legibilidade e a quantidade de estruturas necessárias em SQL e Cypher, entre o banco de dados relacional PostgreSQL e o banco de dados orientado a grafos Neo4j, aplicados a um sistema de recomendação no domínio do comércio eletrônico.

Para alcançar esse objetivo, são definidos os seguintes objetivos específicos:

- Realizar levantamento bibliográfico sobre sistemas de recomendação, abordando filtragem colaborativa e filtragem baseada em conteúdo, bem como os fundamentos teóricos de bancos de dados relacionais e bancos de dados NoSQL orientados a grafos;
- Modelar e estruturar as bases de dados a partir do conjunto de dados público da Olist, plataforma brasileira de comércio eletrônico que disponibiliza dados anonimizados de pedidos¹, adaptando-o aos paradigmas relacional e orientado a grafos, de modo que as duas modelagens utilizem informações equivalentes;
- Desenvolver rotinas de carga e ingestão (ETL) em linguagem Python para tratamento, conversão e inserção dos dados nas duas ferramentas utilizadas;
- Implementar cinco cenários de consulta em SQL e Cypher, contemplando diferentes relações entre clientes, pedidos, produtos, categorias, níveis de conexão e caminhos entre entidades;
- Definir e executar um procedimento de benchmark para comparar os tempos de resposta das consultas nos dois bancos de dados;
- Analisar os resultados obtidos por meio de indicadores descritivos de desempenho e da comparação da estrutura das consultas escritas em SQL e Cypher.

1.3 Justificativa

A escolha da tecnologia de armazenamento e gerenciamento de dados influencia a forma como um sistema de software organiza, consulta e mantém suas informações. No contexto do comércio eletrônico, sistemas de recomendação dependem da análise de relações entre clientes, pedidos, produtos, categorias e vendedores. Por isso, comparar diferentes paradigmas de banco de dados nesse tipo de aplicação pode contribuir para decisões técnicas relacionadas à modelagem e à consulta dos dados.

Este trabalho se justifica pelo interesse em avaliar, sobre uma mesma base de dados, o comportamento de dois paradigmas distintos de persistência: o modelo relacional, representado pelo PostgreSQL, e o modelo orientado a grafos, representado pelo Neo4j. O uso do conjunto de dados público da Olist permite realizar uma comparação reproduzível, com registros anonimizados de pedidos, clientes,

¹ Disponível em: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>.

produtos e vendedores do comércio eletrônico brasileiro. Embora esse conjunto de dados não represente a escala de grandes plataformas comerciais, ele oferece uma base adequada para experimentação acadêmica e para a construção de consultas relacionadas a recomendação.

Além da comparação de desempenho, o trabalho contribui ao observar a forma de escrita das consultas em SQL e Cypher. Comparar a estrutura das consultas permite discutir diferenças de concisão, legibilidade e manutenção entre as abordagens. Dessa forma, os resultados obtidos podem servir como apoio à análise de arquitetos e desenvolvedores que avaliam o uso de bancos relacionais ou orientados a grafos em cenários de recomendação.

1.4 Organização do Trabalho

Após esta introdução, o trabalho apresenta, no Capítulo 2, a fundamentação teórica relacionada a bancos de dados relacionais, bancos de dados NoSQL, bancos de dados orientados a grafos e sistemas de recomendação. O Capítulo 3 descreve a metodologia adotada, incluindo as ferramentas utilizadas, o conjunto de dados da Olist e a arquitetura do experimento. O Capítulo 4 apresenta as consultas implementadas no experimento, a modelagem dos dados nos dois bancos e o procedimento de execução do benchmark. O Capítulo 5 discute os resultados obtidos nos benchmarks, comparando o desempenho e a estrutura das consultas em PostgreSQL e Neo4j. Por fim, o Capítulo 6 apresenta as considerações finais, as limitações do estudo e sugestões de trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

A fundamentação teórica deste trabalho aborda os conceitos relacionados a bancos de dados, bancos de dados relacionais, bancos de dados NoSQL, bancos de dados orientados a grafos, sistemas de recomendação e ferramentas utilizadas na implementação do experimento.

Inicialmente, apresenta-se o papel dos Sistemas Gerenciadores de Bancos de Dados (SGBD) e do modelo relacional, com ênfase na organização dos dados em tabelas, no uso de chaves, na normalização e nas operações de junção. Em seguida, são discutidos os bancos NoSQL e, especificamente, os bancos orientados a grafos, destacando a representação de entidades e relacionamentos. Por fim, são abordados os sistemas de recomendação e suas principais abordagens de filtragem.

2.1 Banco de Dados

Um banco de dados pode ser compreendido como uma coleção organizada de dados relacionados, projetada para representar informações de um domínio de aplicação e permitir operações de armazenamento, consulta, atualização e análise. Um Sistema Gerenciador de Banco de Dados (SGBD), por sua vez, corresponde ao software responsável por administrar esses dados, oferecendo mecanismos de definição, manipulação, segurança, concorrência, integridade e recuperação após falhas (ELMASRI; NAVATHE, 2011).

A principal contribuição dos SGBDs está na separação entre a lógica da aplicação e os detalhes físicos de armazenamento. Em vez de a aplicação manipular diretamente arquivos e estruturas internas de disco, o banco oferece uma camada de abstração capaz de organizar os dados segundo um modelo específico, como o relacional, o documental, o chave-valor, o orientado a colunas ou o orientado a grafos. A escolha desse modelo influencia diretamente a forma como os dados são estruturados, consultados e otimizados pelo mecanismo interno do banco (ELMASRI; NAVATHE, 2011; SADALAGE; FOWLER, 2012).

Essa escolha torna-se relevante em sistemas que manipulam dados interconectados, como plataformas de comércio eletrônico. Nesses ambientes, informações sobre clientes, pedidos, produtos, vendedores e categorias podem ser representadas de diferentes maneiras, e cada representação tende a favorecer determinados padrões de consulta. Por isso, antes de comparar empiricamente

PostgreSQL e Neo4j, é necessário compreender as características estruturais dos modelos de dados que sustentam essas ferramentas.

2.2 Banco de Dados Relacional

O modelo relacional foi proposto por Edgar F. Codd em 1970 e tornou-se um dos paradigmas mais influentes da área de bancos de dados. Nesse modelo, os dados são representados logicamente por relações, implementadas na prática como tabelas compostas por linhas e colunas. Cada linha corresponde a uma tupla, e cada coluna representa um atributo da entidade modelada. A relevância do modelo relacional está na separação entre a visão lógica dos dados e os detalhes físicos de armazenamento, permitindo que o usuário consulte informações sem conhecer os caminhos internos de acesso aos registros (CODD, 1970).

Em sistemas relacionais, a consistência dos dados é mantida por meio de restrições como chaves primárias, chaves estrangeiras, domínios e regras de integridade referencial. Além disso, a normalização organiza os dados em tabelas especializadas, reduzindo redundâncias e minimizando anomalias de inserção, atualização e exclusão. Em uma base de comércio eletrônico, por exemplo, clientes, pedidos, produtos e itens de pedido podem ser armazenados em tabelas separadas, associadas por chaves primárias e estrangeiras.

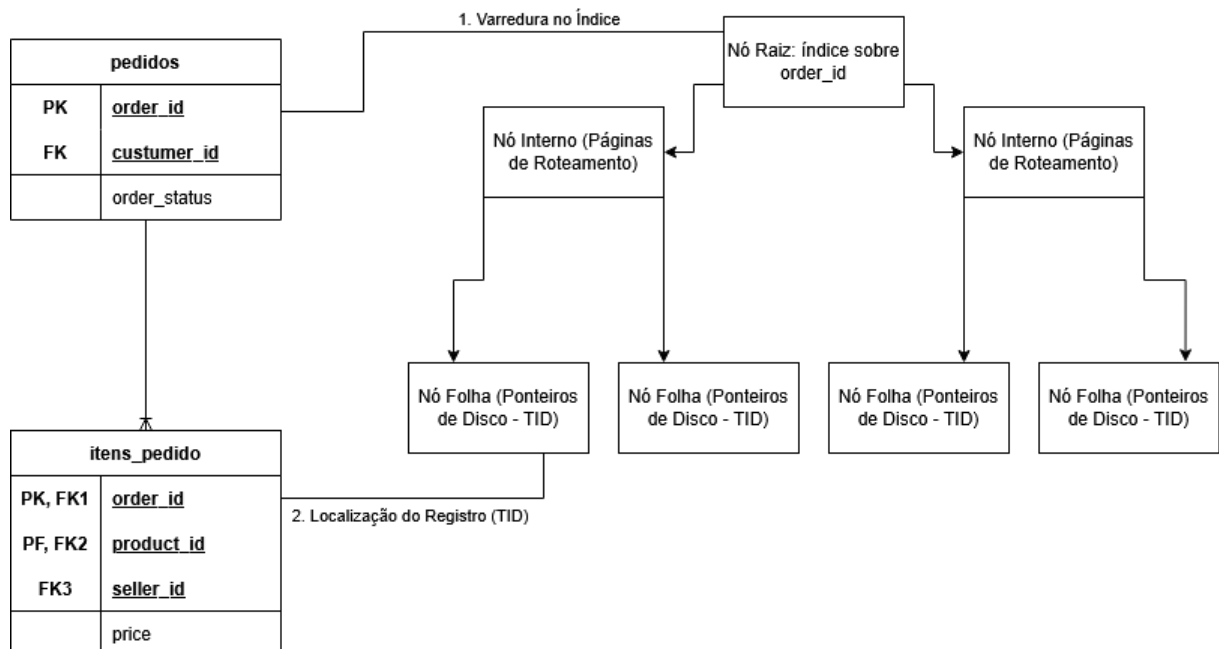
A principal consequência dessa organização é que informações distribuídas em várias tabelas precisam ser combinadas durante a execução das consultas. Essa combinação ocorre por meio de operações de junção, ou `joins`. Em um SGBD relacional, o otimizador de consultas avalia diferentes planos de execução, podendo utilizar estratégias como `nested loop`, `hash join` ou `merge join`, conforme a estrutura da consulta, as estatísticas disponíveis e os índices existentes (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026c).

Como exemplo, podemos considerar uma consulta que precisa identificar quais produtos pertencem a determinado pedido. Em um modelo relacional, a tabela de pedidos pode armazenar o identificador do pedido, enquanto a tabela de itens de pedido armazena os produtos associados a esse identificador. Para recuperar a informação completa, o banco precisa combinar essas tabelas por meio de uma chave comum, como `order_id`. Esse procedimento ilustra a função das junções: reconstruir, no momento da consulta, relações que estão distribuídas entre tabelas distintas.

Em consultas que envolvem poucos relacionamentos entre tabelas e filtros restritivos, como a busca por registros associados a um identificador específico, o uso de índices pode reduzir o número de registros analisados. Em consultas que exigem a combinação de várias tabelas em sequência, o número de relações percorridas aumenta. Neste trabalho, esse número de relações percorridas é tratado como profundidade da consulta, isto é, a quantidade de ligações necessárias para partir de uma entidade inicial e alcançar outras entidades relacionadas.

A Figura 1 apresenta um exemplo conceitual de junção e busca por índice B-Tree no modelo relacional. A parte superior da figura representa a localização de um identificador em uma estrutura de índice, enquanto a parte inferior representa a localização dos registros correspondentes em páginas de dados. O exemplo entre pedidos e itens de pedido mostra que a associação entre tabelas ocorre por meio de valores comuns, como chaves primárias e estrangeiras, e não por uma conexão armazenada diretamente entre os registros.

Figura 1 - Mecanismo de Junção (Join) e Busca por Índice B-Tree no Modelo Relacional



Fonte: Autoria própria

2.3 Banco de Dados NoSQL

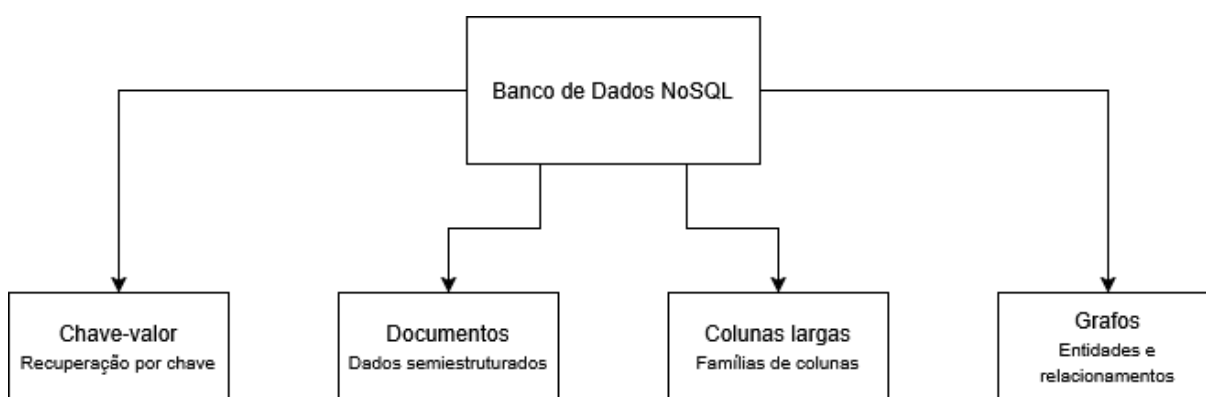
O termo NoSQL, entendido como *Not Only SQL*, designa um conjunto de tecnologias de banco de dados desenvolvidas para atender demandas que nem

sempre se ajustam de forma ideal ao modelo relacional tradicional. Essas demandas incluem flexibilidade de esquema, escalabilidade horizontal, armazenamento de dados semiestruturados e otimização para padrões específicos de consulta. Portanto, NoSQL não corresponde a uma única tecnologia, mas a uma família de modelos de dados com características distintas (SADALAGE; FOWLER, 2012).

Entre as principais categorias de bancos NoSQL estão os bancos chave-valor, orientados a documentos, orientados a colunas largas e orientados a grafos. Bancos chave-valor são otimizados para recuperação simples por chave; bancos documentais armazenam estruturas semiestruturadas, como JSON; bancos orientados a colunas largas favorecem determinados tipos de processamento distribuído; e bancos orientados a grafos priorizam a representação explícita de entidades e relacionamentos.

A Figura 2 sintetiza as principais categorias de bancos NoSQL, evidenciando que esse termo abrange diferentes modelos de dados. Cada categoria organiza e consulta as informações de modo distinto, o que torna a escolha da tecnologia dependente do tipo de dado armazenado e do padrão de consulta esperado.

Figura 2 - Principais Categorias de Bancos de Dados NoSQL



Fonte: Autoria própria

A discussão sobre NoSQL também pode envolver sistemas distribuídos e compromissos entre consistência, disponibilidade e tolerância a particionamento, associados ao Teorema CAP, sigla para *Consistency, Availability and Partition tolerance*. Entretanto, no escopo deste trabalho, o ponto mais relevante está na adequação do modelo de dados ao tipo de consulta analisado, especialmente no caso dos bancos orientados a grafos.

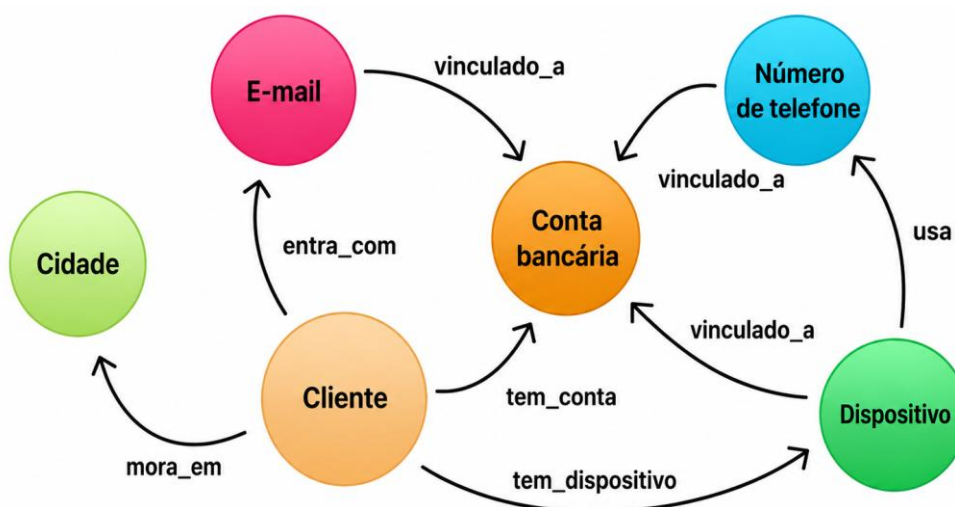
Nesse sentido, os bancos orientados a grafos representam uma categoria específica dentro do conjunto NoSQL. Eles se destacam por modelar entidades e relacionamentos de forma explícita, característica que os diferencia de modelos baseados apenas em tabelas, documentos, chaves ou colunas largas.

2.4 Banco de Dados Orientado a Grafos

Bancos de dados orientados a grafos representam informações por meio de nós e relacionamentos. Os nós correspondem às entidades do domínio, enquanto os relacionamentos representam as conexões entre essas entidades. Essa forma de modelagem é especialmente adequada a problemas nos quais a informação relevante não está apenas nos registros isolados, mas no modo como eles se conectam. Sistemas de recomendação, redes sociais, detecção de fraude, análise de rotas, análise de vínculos e descoberta de caminhos são exemplos de domínios associados a grafos (ROBINSON; WEBBER; EIFREM, 2015).

A Figura 3 apresenta um exemplo de representação em grafo para dados relacionados. No exemplo, entidades como cliente, cidade, e-mail, conta bancária, número de telefone e dispositivo são representadas como nós, enquanto as conexões entre essas entidades são representadas por relacionamentos direcionados, como *mora_em*, *entra_com*, *tem_conta*, *tem_dispositivo*, *usa* e *vinculado_a*. Essa representação evidencia que o valor do modelo não está apenas nos dados isolados, mas também nas relações entre eles.

Figura 3 - Exemplo de Representação em Grafo para Dados Relacionados



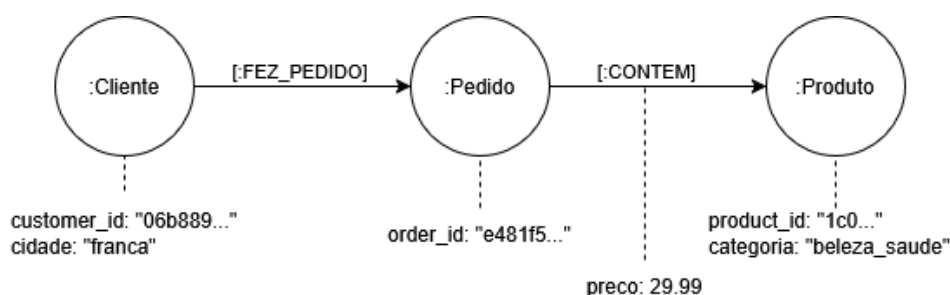
Fonte: Adaptado de Marketing Deal (2023)

A estrutura apresentada na Figura 3 ilustra como um grafo permite percorrer caminhos entre entidades que não estão necessariamente ligadas de forma direta. Por exemplo, um cliente pode estar relacionado a uma conta bancária, a um dispositivo, a um número de telefone e a um e-mail. A partir dessas conexões, torna-se possível identificar vínculos indiretos, analisar relacionamentos sucessivos e representar dados altamente conectados de forma visual e consultável. Esse tipo de estrutura está relacionado a aplicações como detecção de fraude, análise de identidade, recomendação e descoberta de padrões em redes de dados.

O Neo4j, utilizado neste trabalho como representante do paradigma de grafos, adota o modelo de Grafo de Propriedades Rotuladas, ou *Labeled Property Graph* (LPG). Nesse modelo, nós podem receber rótulos que indicam seu tipo, como `:Cliente`, `:Pedido`, `:Produto` e `:Vendedor`; relacionamentos possuem direção e tipo, como `[FEZ_PEDIDO]` e `[CONTEM]`; e tanto nós quanto relacionamentos podem armazenar propriedades em formato chave-valor. Essa estrutura permite que características das entidades e das conexões sejam representadas diretamente no grafo.

A Figura 4 ilustra a estrutura conceitual de um Grafo de Propriedades Rotuladas. Nela, os nós representam entidades, os relacionamentos representam conexões direcionadas entre essas entidades e as propriedades armazenam informações adicionais sobre nós ou relacionamentos.

Figura 4 - Estrutura Conceitual de um Grafo de Propriedades Rotuladas (LPG)



Fonte: Autoria própria

A diferença central em relação ao modelo relacional está no modo de acesso aos relacionamentos. Em um banco relacional, as associações entre tabelas são reconstruídas durante a consulta por meio de junções apoiadas por chaves e, quando disponíveis, por índices. Em um banco orientado a grafos, os índices podem ser

utilizados para localizar nós iniciais, mas a navegação subsequente ocorre por meio dos relacionamentos armazenados no próprio grafo.

Essa navegação é chamada de travessia. Neste contexto, travessia significa percorrer uma sequência de relacionamentos entre nós, partindo de uma entidade inicial e alcançando outras entidades conectadas. Por exemplo, uma travessia pode sair de um cliente, passar pelos pedidos realizados por ele e chegar aos produtos associados a esses pedidos.

Essa característica é conhecida como adjacência livre de índice (*index-free adjacency*). Ela não torna toda consulta automaticamente mais rápida, pois o desempenho também depende da quantidade de relacionamentos percorridos, da seletividade dos filtros e da presença de nós muito conectados. Entretanto, em consultas cujo foco é percorrer relações sucessivas entre entidades, a representação explícita dos relacionamentos pode reduzir a necessidade de reconstrução das conexões por meio de junções tabulares.

2.5 Sistemas de Recomendação

Sistemas de Recomendação (SR) são mecanismos computacionais utilizados para sugerir itens potencialmente relevantes a usuários, com base em dados de preferência, histórico de interação, similaridade entre perfis ou características dos próprios itens. Esses sistemas tornaram-se especialmente importantes em plataformas de comércio eletrônico, nas quais o volume de produtos disponíveis pode dificultar a busca manual e a tomada de decisão por parte do consumidor (RICCI; ROKACH; SHAPIRA, 2011; AGGARWAL, 2016).

De forma geral, um sistema de recomendação procura estimar a utilidade de um item ainda não consumido por determinado usuário. Essa estimativa pode ser construída a partir de avaliações explícitas, como notas e comentários, ou de sinais implícitos, como cliques, visualizações, adições ao carrinho e compras. No contexto de comércio eletrônico, o histórico de compras pode assumir papel relevante, pois permite observar relações entre clientes, produtos e eventos de consumo.

Uma forma clássica de representar o problema de recomendação é por meio de uma matriz de interação entre usuários e itens. Nessa matriz, as linhas representam usuários, as colunas representam itens e as células indicam alguma forma de interação, como compra, avaliação ou interesse observado. Como cada usuário interage apenas com uma parcela do catálogo total, essa matriz tende a ser

esparsa, exigindo técnicas capazes de inferir relações ainda não observadas a partir dos dados disponíveis (RICCI; ROKACH; SHAPIRA, 2011).

A Figura 5 ilustra uma matriz de interação usuário-item. As marcações indicam produtos com os quais determinados usuários já interagiram, enquanto a recomendação inferida representa um item que pode ser sugerido ao usuário-alvo com base em padrões de interação observados entre usuários ou produtos.

Figura 5 - Matriz de Interação e Mecanismo de Dedução por Filtragem Colaborativa

	Produto 1	Produto 2	Produto 3	Produto 4
Usuário A	X	X		
Usuário B	X		X	X
Usuário C			X	
Usuário Alvo	X	X		

Recomendação Inferida

Fonte: Autoria própria

No domínio de e-commerce, as recomendações podem assumir diferentes formas, como sugerir produtos semelhantes aos já comprados, indicar itens frequentemente adquiridos em conjunto ou apresentar produtos consumidos por clientes com perfis parecidos. Essas estratégias dependem da exploração de relações entre usuários, pedidos, produtos e categorias. Por isso, o desempenho de um sistema de recomendação pode ser influenciado tanto pela técnica de recomendação adotada quanto pela forma como essas relações são armazenadas e consultadas.

2.6 Filtragem Colaborativa

A filtragem colaborativa é uma das abordagens mais consolidadas em sistemas de recomendação. Seu princípio central é que usuários que apresentaram comportamentos semelhantes no passado tendem a ter interesses semelhantes no futuro. Em vez de depender exclusivamente da descrição interna dos itens, essa abordagem utiliza o histórico coletivo de interações para identificar padrões de similaridade entre usuários ou entre produtos (RICCI; ROKACH; SHAPIRA, 2011; AGGARWAL, 2016).

Essa técnica é particularmente relevante em comércio eletrônico, pois o histórico de compras permite observar relações de afinidade entre consumidores e produtos. Por exemplo, se diversos clientes que compraram um determinado item também compraram outro produto, o sistema pode inferir uma associação entre esses itens. Da mesma forma, se dois clientes possuem históricos de compra semelhantes, produtos adquiridos por um deles podem ser recomendados ao outro. Essas inferências são construídas a partir das interações registradas, sem exigir necessariamente uma descrição semântica detalhada de cada produto.

A filtragem colaborativa pode ser classificada conforme duas perspectivas principais. Na abordagem baseada em usuários, o sistema identifica consumidores com histórico semelhante ao do usuário-alvo e recomenda itens consumidos por esses vizinhos. Na abordagem baseada em itens, o foco está na relação entre os próprios produtos, identificando itens frequentemente consumidos em conjunto ou por públicos semelhantes. Ambas dependem da exploração de relações entre usuários, itens e eventos de interação, o que torna o armazenamento dessas relações um fator relevante para sua consulta e processamento.

Apesar de sua utilidade, a filtragem colaborativa possui limitações. Uma delas é o problema do início frio (*cold start*), que ocorre quando novos usuários ou novos produtos ainda não possuem interações suficientes para gerar recomendações confiáveis. Outra limitação é a esparsidade da matriz usuário-item: quanto menor a quantidade de interações observadas, mais difícil se torna identificar vizinhos ou produtos semelhantes com alto grau de confiança (RICCI; ROKACH; SHAPIRA, 2011; AGGARWAL, 2016).

2.7 Filtragem Baseada em Conteúdo

A filtragem baseada em conteúdo adota uma lógica diferente da filtragem colaborativa. Em vez de utilizar principalmente o comportamento coletivo dos usuários, essa abordagem recomenda itens semelhantes àqueles pelos quais o usuário já demonstrou interesse. Para isso, considera características dos próprios produtos, como categoria, marca, descrição, preço, especificações técnicas ou outros metadados disponíveis no catálogo (RICCI; ROKACH; SHAPIRA, 2011; AGGARWAL, 2016).

Em ambientes de comércio eletrônico, a filtragem baseada em conteúdo é relevante porque os produtos costumam possuir atributos descritivos que podem ser

usados para estimar similaridade. Se um usuário compra com frequência itens de determinada categoria, o sistema pode priorizar produtos pertencentes a categorias próximas ou equivalentes. Diferentemente da filtragem colaborativa, essa abordagem não depende exclusivamente de outros usuários com comportamento semelhante, pois utiliza informações internas dos itens para orientar a recomendação.

Uma vantagem importante da filtragem baseada em conteúdo é sua utilidade em situações nas quais há poucos dados coletivos disponíveis. Produtos novos, por exemplo, podem ainda não ter sido adquiridos por muitos clientes, mas já possuem categoria, descrição e outros atributos cadastrados. Esses metadados permitem que o sistema os recomende com base na proximidade em relação ao perfil de consumo do usuário. Essa característica ajuda a reduzir parte do problema de início frio associado a novos itens (AGGARWAL, 2016).

Entretanto, a abordagem baseada em conteúdo também apresenta limitações. Como tende a recomendar itens semelhantes aos que o usuário já consumiu, ela pode reduzir a diversidade das recomendações e limitar a descoberta de produtos fora do padrão histórico do consumidor. Por esse motivo, sistemas reais frequentemente combinam diferentes estratégias, formando recomendações híbridas que utilizam tanto padrões coletivos de interação quanto características internas dos itens.

2.8 Ferramentas Utilizadas

Foram empregadas ferramentas capazes de representar os dois paradigmas comparados e permitir a automação das etapas de carga, consulta e medição de desempenho. O PostgreSQL foi utilizado como representante do modelo relacional; o Neo4j, como representante do modelo orientado a grafos; e a linguagem Python, com bibliotecas de manipulação de dados e conexão com bancos, como apoio à preparação dos dados e à execução dos testes.

2.8.1 PostgreSQL

O PostgreSQL é um Sistema Gerenciador de Banco de Dados Objeto-Relacional de código aberto, utilizado em aplicações que exigem integridade, estabilidade e suporte à linguagem SQL. Neste trabalho, ele foi selecionado como representante do modelo relacional por permitir a modelagem normalizada dos dados

e por oferecer mecanismos consolidados de consulta, indexação e otimização (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026a).

Entre os recursos relevantes do PostgreSQL para este trabalho estão o suporte a chaves primárias e estrangeiras, restrições de integridade, consultas SQL, otimizador de consultas e índices. Os índices são estruturas auxiliares utilizadas para acelerar a localização de registros, reduzindo a necessidade de percorrer toda a tabela em determinadas consultas. Entre os tipos de índice disponíveis, o B-Tree é um dos mais comuns e é adequado para buscas por igualdade, intervalos e ordenação em colunas indexadas (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026b).

No contexto de consultas relacionais, o PostgreSQL combina o uso de tabelas, chaves e índices com o otimizador de consultas. Esse otimizador avalia diferentes estratégias para executar uma consulta, considerando fatores como filtros, junções, índices disponíveis e estatísticas das tabelas. Essas características tornam o PostgreSQL adequado como base de comparação para consultas estruturadas sobre dados tabulares.

2.8.2 Neo4j e Cypher

O Neo4j é um banco de dados orientado a grafos nativo, baseado no modelo de Grafo de Propriedades Rotuladas. Nesse modelo, dados são representados por nós, relacionamentos e propriedades. Essa forma de representação permite modelar diretamente conexões entre entidades, o que é relevante em problemas nos quais a análise das relações possui papel central.

A linguagem Cypher é utilizada para consultar e manipular dados no Neo4j. Sua sintaxe permite expressar padrões de conexão entre nós e relacionamentos de forma declarativa. Por exemplo, uma consulta pode indicar um caminho entre entidades por meio de uma sequência de nós e relacionamentos, tornando explícita a estrutura que será percorrida no grafo.

O Neo4j e a linguagem Cypher são relevantes porque permitem representar e consultar dados de recomendação a partir de conexões entre entidades. Essa característica possibilita comparar a escrita das consultas em Cypher com a escrita das consultas SQL no PostgreSQL.

2.8.3 Linguagem Python e Ecossistema de Dados

Python é uma linguagem de programação amplamente utilizada em tarefas de automação, análise de dados, integração de sistemas e desenvolvimento de scripts. Sua adoção em projetos de dados está relacionada à disponibilidade de bibliotecas para leitura de arquivos, manipulação de tabelas, conexão com bancos de dados e execução de rotinas automatizadas.

Entre essas bibliotecas, destaca-se o pandas, voltado à manipulação e análise de dados tabulares. Ele permite carregar arquivos em formato CSV, selecionar colunas, tratar valores ausentes, transformar estruturas de dados e preparar informações para etapas posteriores de processamento. Esses recursos são úteis em processos de Extração, Transformação e Carga (ETL), nos quais dados brutos precisam ser organizados antes de serem inseridos em bancos de dados.

Além do pandas, bibliotecas de conexão com bancos permitem que scripts Python executem comandos SQL ou consultas em bancos orientados a grafos. Esse tipo de integração possibilita automatizar a carga dos dados, disparar consultas, controlar repetições e registrar resultados de execução. Dessa forma, Python atua como uma camada de apoio para experimentos que dependem da preparação de dados e da comparação controlada entre diferentes tecnologias.

3 METODOLOGIA

Este capítulo descreve os procedimentos metodológicos adotados para atingir os objetivos propostos. São apresentadas as ferramentas e tecnologias utilizadas, o ambiente de execução, o conjunto de dados da Olist e a arquitetura do experimento comparativo.

3.1 Ferramentas e Tecnologias

Para a execução do experimento, foram utilizadas ferramentas de código aberto ou com licenciamento gratuito para uso acadêmico. A pilha tecnológica adotada foi composta pelos seguintes elementos:

- Banco de Dados Relacional: foi utilizado o PostgreSQL. A escolha deste Sistema Gerenciador de Banco de Dados Objeto-Relacional (SGBDOR) justifica-se pelo suporte à linguagem SQL, pela possibilidade de modelagem relacional normalizada, pelo uso de restrições de integridade e pela criação de índices B-Tree sobre colunas utilizadas nas consultas;
- Banco de Dados Orientado a Grafos: foi adotado o Neo4j Desktop. A escolha está relacionada ao suporte ao modelo de Grafo de Propriedades Rotuladas (LPG), à representação explícita de nós e relacionamentos e à utilização da linguagem declarativa Cypher;
- Orquestração, ETL e Benchmark: a linguagem Python foi utilizada para automatizar as rotinas de preparação, carga e execução dos testes. A biblioteca pandas foi empregada no tratamento dos arquivos CSV, enquanto as bibliotecas psycopg2 e neo4j foram utilizadas para conexão com o PostgreSQL e com o Neo4j, respectivamente;
- Ambiente de Desenvolvimento e Modelagem: a escrita dos códigos ocorreu no Visual Studio Code (VS Code). Para a criação dos diagramas de modelagem, como o Diagrama Entidade-Relacionamento (DER) e o modelo de Grafo de Propriedades Rotuladas (LPG), foi utilizada a ferramenta diagrams.net.

3.1.1 Ambiente de Execução

O experimento foi executado localmente em uma única máquina. Neste trabalho, o isolamento de hardware significa que os dois SGBDs foram avaliados no

mesmo equipamento, utilizando os mesmos recursos físicos de processamento, memória e armazenamento. Essa escolha reduz a interferência de fatores externos, como variações de rede, e permite que a comparação se concentre no comportamento das consultas em cada banco de dados.

A estação de trabalho utilizada como servidor local de banco de dados e executor do benchmark apresentou as seguintes especificações de hardware e software:

- Equipamento: Notebook modelo Acer Nitro V15 (ANV15-52-790M);
- Processador (CPU): Intel Core i7-13620H de 13ª geração, com 10 núcleos e 16 threads, e frequência turbo máxima de até 4.9 GHz;
- Memória RAM: 16 GB com tecnologia DDR5;
- Armazenamento Secundário: Unidade de Estado Sólido (SSD) de 512 GB com interface NVMe;
- Aceleração Gráfica (GPU): NVIDIA GeForce RTX 4060 com 8 GB GDDR6, não utilizada diretamente no processamento do benchmark;
- Sistema Operacional: Microsoft Windows 11 Pro, arquitetura de 64 bits.

3.2 Objeto de Estudo

O objeto de estudo selecionado foi o conjunto de dados público Brazilian E-Commerce Public Dataset by Olist, disponibilizado na plataforma Kaggle (OLIST, [s. d.]). Esse dataset reúne informações anonimizadas de pedidos realizados na plataforma da Olist entre os anos de 2016 e 2018, incluindo dados sobre clientes, pedidos, itens de pedido, produtos, vendedores, pagamentos, avaliações e geolocalização.

A escolha desse conjunto de dados se justifica por três características principais:

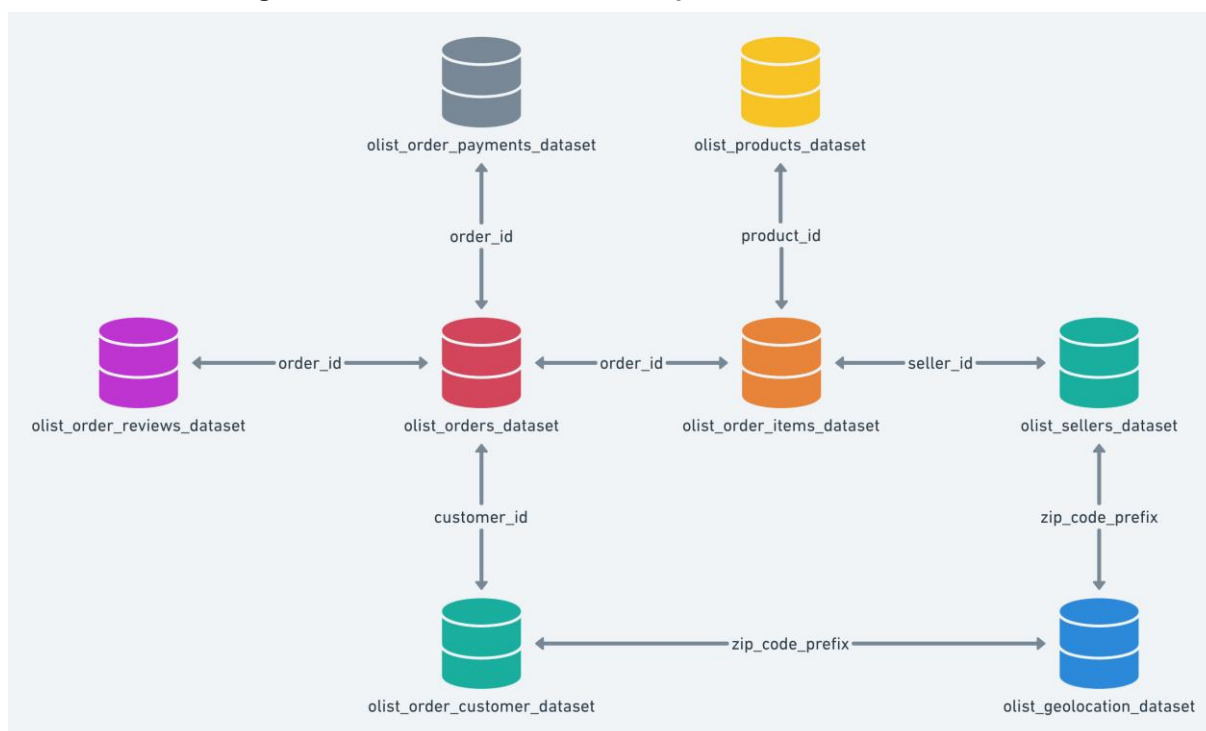
1. Reprodutibilidade: por se tratar de um repositório público e documentado, o dataset pode ser baixado e utilizado em novos experimentos, permitindo a reprodução das etapas de carga, modelagem e consulta;
2. Volume e variedade dos dados: o conjunto contém mais de 100.000 pedidos associados a clientes, produtos, vendedores e outras entidades do domínio de

comércio eletrônico. Essa composição permite construir consultas que envolvem múltiplas tabelas e relacionamentos;

3. Estrutura relacional: os dados são disponibilizados em arquivos no formato Comma-Separated Values (CSV), organizados por entidades e relacionados por identificadores. Essa estrutura permite carregar os dados no modelo relacional e também adaptá-los para uma representação em grafo, mantendo a mesma origem de dados para os dois paradigmas.

Para mapear a origem das informações antes da etapa de transformação, a Figura 6 apresenta a estrutura relacional simplificada do dataset da Olist e os principais relacionamentos entre seus arquivos.

Figura 6 - Estrutura Relacional Simplificada do Dataset Olist



Fonte: Kaggle (s.d.)

3.3 Arquitetura do Experimento

A comparação foi organizada em um fluxo de trabalho dividido em etapas. As duas trilhas partiram da mesma fonte de dados brutos, composta pelos arquivos CSV da Olist, e divergiram na tecnologia de persistência utilizada: PostgreSQL para o modelo relacional e Neo4j para o modelo orientado a grafos.

1. Extração, Transformação e Carga (ETL): utilizando a linguagem Python e a biblioteca pandas, os arquivos CSV da Olist foram lidos e preparados para uso nos dois bancos de dados. O tratamento incluiu a seleção das colunas necessárias ao experimento, a padronização dos atributos utilizados nas cargas e a remoção de registros sem identificadores essenciais para a criação das relações entre entidades. Após esse tratamento, os dados foram adequados às estruturas relacional e orientada a grafos.
2. Carga no PostgreSQL: o script `carga_postgres.py` foi responsável por criar as tabelas relacionais e inserir os dados tratados no PostgreSQL. A organização relacional utilizou identificadores como chaves primárias e estrangeiras para manter associações entre registros, como clientes, pedidos, produtos e itens de pedido.

A Tabela 1 apresenta exemplos de estruturas relacionais criadas no PostgreSQL a partir dos arquivos da Olist. Ela demonstra como os dados foram organizados em tabelas e como identificadores como `customer_id`, `order_id` e `product_id` foram utilizados para relacionar clientes, pedidos, produtos e itens de pedido.

Tabela 1 - Exemplos de Estruturas Relacionais Criadas no PostgreSQL

Tabela	Chave principal	Campos exemplificados	Origem no dataset
clientes	<code>customer_id</code>	<code>customer_id</code> , <code>customer_city</code>	<code>olist_customers_dataset.csv</code>
pedidos	<code>order_id</code>	<code>order_id</code> , <code>customer_id</code>	<code>olist_orders_dataset.csv</code>
produtos	<code>product_id</code>	<code>product_id</code> , <code>product_category_name</code>	<code>olist_products_dataset.csv</code>
Itens_pedido	<code>order_id</code> , <code>product_id</code> , <code>seller_id</code>	<code>order_id</code> , <code>product_id</code> , <code>seller_id</code> , <code>price</code>	<code>olist_order_items_dataset.csv</code>

Fonte: Autoria própria

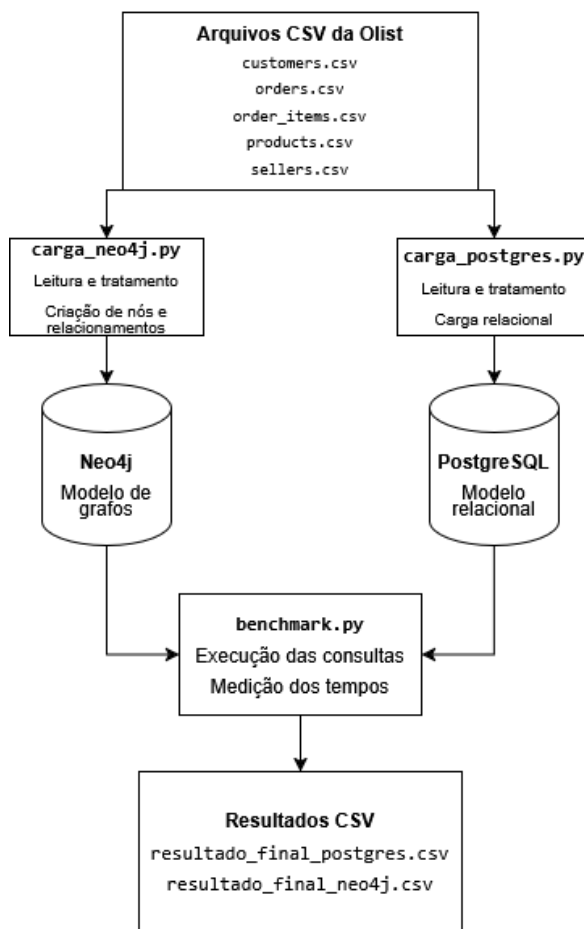
3. Carga no Neo4j: o script `carga_neo4j.py` foi responsável por converter os registros selecionados em nós e relacionamentos no Neo4j. Clientes, pedidos

e produtos foram representados como nós, enquanto as relações entre eles foram representadas por relacionamentos direcionados. Conforme ilustrado na Figura 4, o relacionamento `[ :FEZ_PEDIDO ]` indica que um cliente realizou determinado pedido, enquanto `[ :CONTEM ]` indica que um pedido contém determinado produto. Quando aplicável, atributos da associação, como preço, foram armazenados como propriedades do relacionamento.

4. Execução do benchmark: o script `benchmark.py` foi utilizado para executar as consultas nos dois bancos de dados e registrar os tempos de resposta. Para cada execução, foram utilizados identificadores existentes na base carregada, como chaves de clientes ou produtos, evitando consultas sobre registros inexistentes. A medição do tempo foi realizada na camada de aplicação, por meio da função `time.perf_counter()` da linguagem Python.
5. Coleta e análise dos resultados: os tempos de execução foram salvos em arquivos CSV de resultado, permitindo a tabulação dos dados e o cálculo dos indicadores descritivos utilizados na análise. Antes das medições consideradas finais, foram executadas rodadas de aquecimento, descartadas da análise, com o objetivo de reduzir o efeito da primeira execução sobre os tempos registrados, procedimento recomendado em avaliações de desempenho computacional (JAIN, 1991).

A Figura 7 apresenta a arquitetura geral do experimento. Nela, os arquivos CSV da Olist aparecem como entrada do processo; os scripts `carga_postgres.py` e `carga_neo4j.py` direcionam os dados para PostgreSQL e Neo4j; o script `benchmark.py` executa as consultas nos dois bancos; e os resultados são armazenados em arquivos CSV para posterior análise.

Figura 7 - Arquitetura Geral e Fluxo de Execução do Experimento



Fonte: Autoria própria

4 DESENVOLVIMENTO

Este capítulo apresenta as consultas implementadas no experimento, a modelagem dos dados nos dois bancos utilizados e o procedimento de execução do benchmark. Inicialmente, são descritos os cenários de consulta avaliados, pois eles definem quais relações entre clientes, pedidos, produtos e vendedores precisam ser representadas nos modelos PostgreSQL e Neo4j. Em seguida, são apresentados os dois modelos de dados e a forma como as consultas foram executadas para comparação de desempenho e expressividade.

4.1 Consultas Implementadas no Experimento

As consultas implementadas no experimento foram definidas para representar diferentes formas de recomendação e análise de relacionamento em um cenário de comércio eletrônico. Elas utilizam entidades presentes no dataset da Olist, como clientes, pedidos, produtos, vendedores e itens de pedido, e foram escritas em versões equivalentes para PostgreSQL, por meio da linguagem SQL, e para Neo4j, por meio da linguagem Cypher.

Neste trabalho, a profundidade de uma consulta corresponde à quantidade de relações percorridas entre entidades para chegar ao resultado esperado. Consultas de baixa profundidade percorrem poucas relações diretas, como produto-pedido ou cliente-pedido-produto. Consultas de profundidade intermediária ampliam esse percurso para incluir entidades conectadas por produtos ou clientes em comum. Consultas de maior profundidade percorrem cadeias mais longas de relações, exigindo sucessivas junções no PostgreSQL ou travessias mais extensas no Neo4j.

A Tabela 2 apresenta os objetivos de cada consulta e as principais estruturas envolvidas em sua execução. No PostgreSQL, essas estruturas aparecem como tabelas relacionadas por chaves primárias e estrangeiras. No Neo4j, as mesmas relações são representadas por nós e relacionamentos.

Tabela 2 - Síntese das Consultas Implementadas no Benchmark

Consulta	Objetivo	Entrada principal	Estruturas no PostgreSQL	Estruturas no Neo4j
Q1	Recomendar produtos comprados por clientes que também	product_id	itens_pedido, pedidos,	:Produto, :Pedido, :Cliente,

Consulta	Objetivo	Entrada principal	Estruturas no PostgreSQL	Estruturas no Neo4j
	compraram um produto de referência		clientes, produtos	[:CONTEM], [:FEZ_PEDIDO]
Q2	Identificar produtos comprados no mesmo pedido	product_id	itens_pedido	:Produto, :Pedido, [:CONTEM]
Q3	Recomendar produtos com base em clientes da mesma cidade e categorias já consumidas	customer_id	clientes, pedidos, itens_pedido, produtos	:Cliente, :Pedido, :Produto, [:FEZ_PEDIDO], [:CONTEM]
Q4	Aumentar progressivamente a profundidade das relações analisadas	customer_id	pedidos, itens_pedido, produtos	:Cliente, :Pedido, :Produto, [:FEZ_PEDIDO], [:CONTEM]
Q5	Encontrar o menor caminho entre dois clientes	dois customer_id	pedidos, itens_pedido, consulta recursiva com WITH RECURSIVE	:Cliente, :Pedido, :Produto, relacionamentos variáveis e shortestPath

Fonte: Autoria própria

Consulta Q1 - Filtragem Colaborativa Simples

A Consulta Q1 parte de um produto de referência e busca outros produtos comprados por clientes que também adquiriram esse produto. No modelo relacional, a consulta envolve as tabelas `itens_pedido`, `pedidos`, `clientes` e `produtos`, combinadas por meio de junções. No modelo de grafo, a mesma lógica é representada por um percurso entre nós `:Produto`, `:Pedido` e `:Cliente`, utilizando os relacionamentos `[ :CONTEM ]` e `[ :FEZ_PEDIDO ]`.

Nesse contexto, o percurso da consulta pode ser entendido como uma sequência de relações entre entidades. A consulta parte de um produto, chega aos

pedidos que contêm esse produto, identifica os clientes associados a esses pedidos e retorna outros produtos comprados por esses mesmos clientes. Assim, a explicação substitui o uso isolado da expressão “saltos lógicos”, que poderia gerar ambiguidade.

Consulta Q2 - Análise de Cesta de Compras

A Consulta Q2 identifica produtos frequentemente comprados no mesmo pedido. Ela representa uma análise de coocorrência de itens, comum em sistemas de recomendação baseados em cesta de compras. No PostgreSQL, a consulta utiliza principalmente a tabela `itens_pedido`, comparando produtos que aparecem associados ao mesmo `order_id`. No Neo4j, a mesma lógica percorre um nó `:Pedido` conectado a diferentes nós `:Produto` por meio do relacionamento `[ :CONTEM ]`.

Consulta Q3 - Recomendação Híbrida por Perfil e Geografia

A Consulta Q3 combina informações de perfil do cliente e características dos produtos. Ela parte de um cliente de referência, identifica outros clientes da mesma cidade e recomenda produtos pertencentes a categorias pelas quais o cliente de referência já demonstrou interesse. No PostgreSQL, a consulta envolve as tabelas `clientes`, `pedidos`, `itens_pedido` e `produtos`. No Neo4j, a consulta utiliza nós `:Cliente`, `:Pedido` e `:Produto`, além das propriedades `cidade` e `categoria`.

Essa consulta foi classificada como híbrida porque combina duas fontes de informação: o comportamento de compra de clientes semelhantes e os metadados dos produtos, representados pela categoria.

Consulta Q4 - Teste Escalonado de Profundidade

A Consulta Q4 foi organizada em quatro variações: Q4_G1, Q4_G2, Q4_G3 e Q4_G4. A letra G indica o grau de expansão da consulta, isto é, o nível de relações percorridas a partir do cliente de referência. Essas variações foram utilizadas para observar o comportamento dos dois bancos conforme a profundidade da consulta aumenta.

A versão Q4_G1 representa uma consulta de baixa profundidade, pois busca produtos comprados diretamente pelo cliente de referência. A versão Q4_G2 amplia a análise para clientes conectados por produtos em comum, representando uma profundidade intermediária. As versões Q4_G3 e Q4_G4 aumentam progressivamente a quantidade de relações percorridas, sendo tratadas como consultas de maior profundidade neste experimento.

No PostgreSQL, esse aumento de profundidade exige mais combinações entre as tabelas `pedidos` e `itens_pedido`. No Neo4j, o mesmo aumento é representado por sequências maiores de relacionamentos entre clientes, pedidos e produtos. Dessa forma, a série Q4 permite comparar como os dois modelos se comportam quando a consulta passa de relações diretas para cadeias mais longas de conexão.

Consulta Q5 - Caminho Mais Curto entre Clientes

A Consulta Q5 busca identificar o menor caminho entre dois clientes a partir das conexões indiretas formadas por pedidos e produtos. No PostgreSQL, essa consulta foi implementada por meio de uma consulta recursiva, utilizando `WITH RECURSIVE`, pois o modelo relacional não armazena caminhos entre entidades como conexões diretas. No Neo4j, a mesma lógica foi implementada com a função `shortestPath`, própria da linguagem Cypher para busca de caminhos em grafos.

Essa consulta foi incluída para comparar a expressividade das duas linguagens em um cenário de descoberta de caminhos. No SQL, a formulação exige uma estrutura recursiva. No Cypher, a consulta pode ser expressa diretamente como um caminho entre dois nós `:Cliente`.

4.2 Modelagem de Dados

O conjunto de dados da Olist é disponibilizado em arquivos CSV separados por entidades do domínio de comércio eletrônico. Entre os arquivos utilizados estão dados de clientes, pedidos, itens de pedido, produtos e vendedores. Esses arquivos se relacionam por identificadores como `customer_id`, `order_id`, `product_id` e `seller_id`, permitindo associar clientes aos pedidos realizados, pedidos aos produtos comprados e produtos aos vendedores.

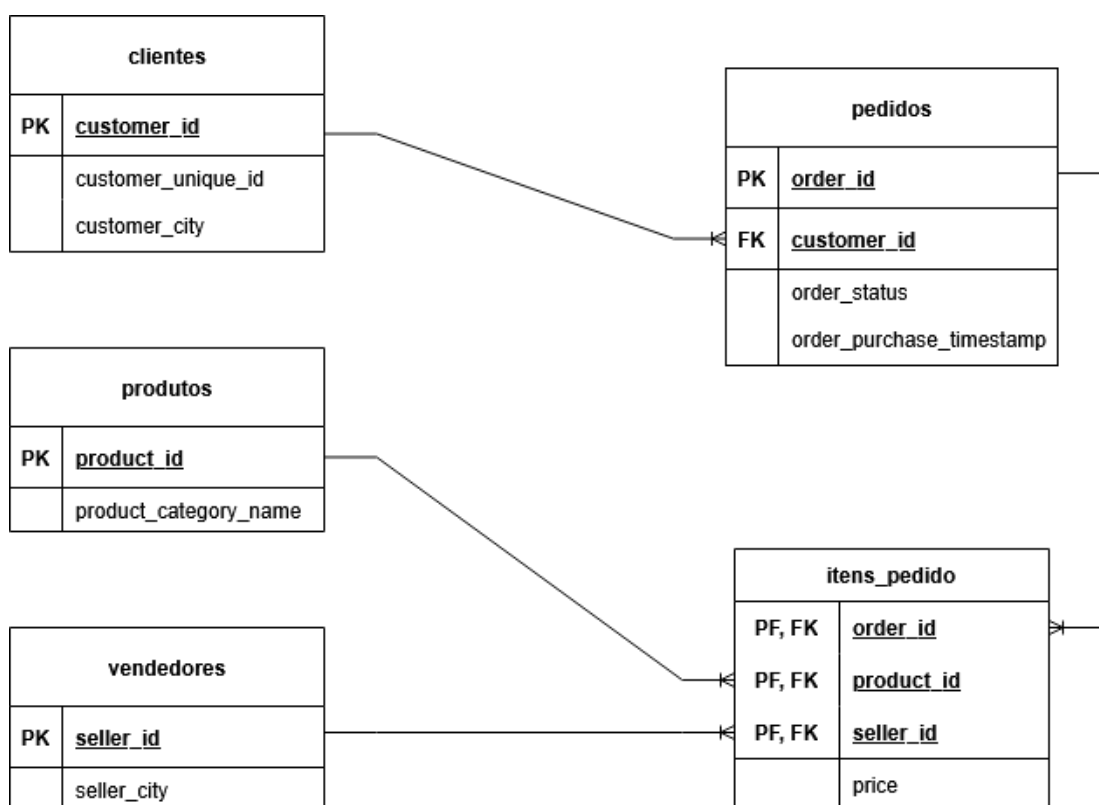
A partir dessa estrutura, os dados foram organizados em dois modelos. No PostgreSQL, os arquivos foram representados por tabelas relacionadas por chaves primárias e estrangeiras. No Neo4j, as principais entidades foram representadas como nós, e as associações entre elas foram representadas como relacionamentos direcionados. Essa dupla modelagem permitiu analisar a mesma origem de dados sob dois paradigmas distintos de persistência e consulta.

4.2.1 Modelo A: Modelo Relacional (PostgreSQL)

O objetivo do Modelo A foi representar os dados da Olist no paradigma relacional, utilizando tabelas, chaves e relacionamentos por identificadores. Essa modelagem permitiu executar as consultas de recomendação em SQL no PostgreSQL, mantendo a organização tabular dos dados.

O modelo relacional foi estruturado a partir de entidades como clientes, produtos, vendedores e pedidos, interligadas pela tabela associativa `itens_pedido`. Essa tabela representa a relação entre pedidos, produtos e vendedores, permitindo resolver associações de cardinalidade muitos-para-muitos. A Figura 8 apresenta o Diagrama Entidade-Relacionamento do Modelo A, mostrando as principais tabelas e os identificadores utilizados para relacioná-las.

Figura 8 - Diagrama Entidade-Relacionamento do Modelo Relacional (PostgreSQL)



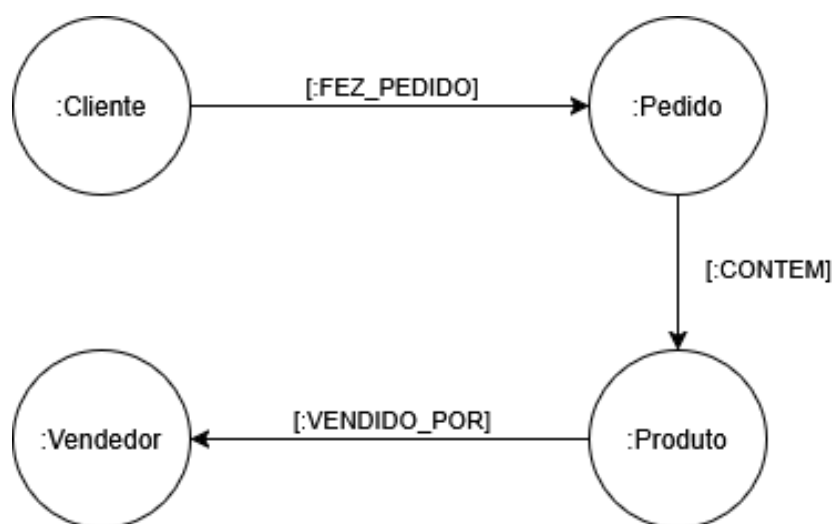
Fonte: Autoria própria

Após a carga dos dados, foram criados índices B-Tree sobre colunas utilizadas como chaves estrangeiras e sobre atributos empregados como filtros nas consultas, como `customer_city` e `product_category_name`. Esses índices foram utilizados para apoiar operações de busca, filtragem e junção entre tabelas, de acordo com os recursos de indexação disponíveis no PostgreSQL.

4.2.2 Modelo B: Modelo de Grafo (Neo4j)

O objetivo do Modelo B foi representar os dados da Olist no paradigma orientado a grafos, utilizando nós, relacionamentos e propriedades. Essa modelagem permitiu executar as consultas de recomendação em Cypher no Neo4j, explorando a representação explícita das conexões entre entidades.

Figura 9 - Modelo de Grafo (Neo4j)



Fonte: Autoria própria

No Modelo B, entidades como clientes, pedidos, produtos e vendedores foram representadas como nós, com rótulos como `:Cliente`, `:Pedido`, `:Produto` e `:Vendedor`. As associações entre essas entidades foram representadas por relacionamentos direcionados. A Figura 9 apresenta o modelo de grafo utilizado no Neo4j, incluindo os principais nós e relacionamentos derivados da base da Olist.

A relação entre cliente e pedido foi representada pelo relacionamento `[FEZ_PEDIDO]`, indicando que determinado cliente realizou um pedido. A relação entre pedido e produto foi representada pelo relacionamento `[CONTEM]`, indicando que determinado pedido contém um produto. A associação entre produto e vendedor foi representada pelo relacionamento `[VENDIDO_POR]`.

A entidade associativa `itens_pedido`, utilizada no modelo relacional, foi representada no grafo por meio do relacionamento `[CONTEM]` entre `:Pedido` e `:Produto`. Atributos dessa associação, como `preco`, foram armazenados como propriedades do relacionamento. Dessa forma, parte da informação que no modelo relacional aparece em uma tabela intermediária passa a ser representada diretamente na conexão entre os nós no modelo de grafos.

Essa organização está relacionada ao conceito de adjacência livre de índice, apresentado na fundamentação teórica. Após a localização de um nó inicial, a consulta pode percorrer os relacionamentos armazenados no grafo para alcançar entidades conectadas. Essa característica foi considerada na modelagem do Neo4j, especialmente para representar relações entre clientes, pedidos, produtos e vendedores.

4.3 Procedimento de Execução do Benchmark

O benchmark foi executado por meio do script `benchmark.py`, responsável por conectar-se ao PostgreSQL e ao Neo4j, executar as consultas equivalentes em SQL e Cypher, medir os tempos de resposta e registrar os resultados em arquivos CSV. O fluxo geral desse processo foi apresentado anteriormente na Figura 7, localizada na seção 3.3 Arquitetura do Experimento.

Tabela 3 - Parâmetros de Execução do Benchmark

Parâmetro	Configuração utilizada	Finalidade
Script de execução	<code>benchmark.py</code>	Executar consultas, medir tempos e gerar arquivos de resultado
Bancos avaliados	PostgreSQL e Neo4j	Comparar o modelo relacional e o modelo orientado a grafos
Entradas das consultas	<code>customer_id</code> e <code>product_id</code> existentes na base	Evitar consultas sobre registros inexistentes
Seleção das entradas	Sorteio de clientes e produtos a partir de listas previamente selecionadas	Reduzir dependência de um único registro da base
Execuções de aquecimento	5 execuções descartadas	Reduzir o efeito da primeira execução sobre a medição final
Repetições padrão	até 50 execuções válidas	Coletar tempos para cálculo de média e desvio-padrão
Repetições em consultas custosas	até 10 execuções válidas para Q4_G4 e Q5	Evitar execuções excessivamente longas

Parâmetro	Configuração utilizada	Finalidade
Medição de tempo	<code>time.perf_counter()</code>	Medir o tempo de execução na camada de aplicação
Timeout no PostgreSQL	120 segundos	Interromper consultas que ultrapassassem o limite definido
Saída dos resultados	arquivos CSV	Registrar banco, consulta, média em ms e desvio-padrão

Fonte: Autoria própria

A Tabela 3 sintetiza os principais parâmetros utilizados na execução do benchmark. Esses parâmetros foram definidos para padronizar a comparação entre os dois bancos de dados, mantendo o mesmo conjunto de consultas, o mesmo ambiente de execução e a mesma lógica de medição.

Antes da execução das consultas, o script selecionou identificadores existentes na base carregada, como clientes e produtos. No caso dos clientes, foram considerados registros associados a mais de um produto; no caso dos produtos, foram considerados itens presentes em mais de um pedido. Esses identificadores foram utilizados como entradas das consultas, evitando execuções sobre dados inexistentes ou pouco representativos.

Durante as repetições, o script sorteou identificadores a partir dessas listas de clientes e produtos. Essa seleção variada de entradas foi utilizada para reduzir a dependência dos resultados em relação a um único cliente ou produto específico. Dessa forma, a medição não ficou concentrada sempre no mesmo registro da base.

A medição do tempo foi realizada na camada de aplicação, utilizando a função `time.perf_counter()` da linguagem Python. Para cada consulta executada com sucesso, o tempo foi convertido para milissegundos e armazenado para cálculo posterior da média e do desvio-padrão.

Antes das execuções consideradas no cálculo final, foram realizadas cinco execuções de aquecimento. Essas execuções foram processadas, mas seus tempos foram descartados. O objetivo foi reduzir o efeito da primeira execução, que pode incluir custos iniciais de conexão, leitura e preparação da consulta.

As consultas de menor profundidade foram executadas com até 50 repetições válidas. Para consultas mais custosas, como Q4_G4 e Q5, o número de repetições foi limitado a 10, reduzindo o tempo total do experimento e evitando execuções excessivamente longas.

No PostgreSQL, foi configurado um limite de execução de 120 segundos por consulta, por meio do parâmetro `statement_timeout`. Quando esse limite era ultrapassado, a execução era interrompida e registrada como falha. Esse tratamento evitou que uma consulta excessivamente longa impedisse a continuidade do experimento.

No Neo4j, também foi aplicado tratamento de exceções na execução das consultas. Caso ocorresse falha durante a execução, o evento era registrado no resultado da respectiva consulta. Em ambos os bancos, falhas ou timeouts foram representados nos arquivos de saída sem média válida, permitindo distinguir consultas concluídas de consultas interrompidas.

Ao final da execução, o script gerou arquivos CSV de resultado para cada banco de dados. Esses arquivos continham o nome do banco, o identificador da consulta, a média dos tempos de execução em milissegundos e o desvio-padrão. Esses dados serviram de base para as tabelas e análises apresentadas no capítulo de resultados.

5 ANÁLISE DOS RESULTADOS E DISCUSSÃO

Este capítulo apresenta e discute os resultados obtidos no benchmark realizado com PostgreSQL e Neo4j. A análise considera os tempos médios de execução, os desvios-padrão e os casos de timeout registrados durante a execução das consultas descritas no capítulo anterior.

Os resultados foram organizados em três grupos: consultas de baixa profundidade, teste de aumento progressivo de profundidade e busca de caminho mais curto entre clientes. Neste trabalho, consultas de baixa profundidade são aquelas que percorrem poucas relações diretas entre entidades, como cliente-pedido-produto ou pedido-produto. Já as consultas de maior profundidade percorrem cadeias mais longas de relações, exigindo mais junções no PostgreSQL ou travessias mais extensas no Neo4j. Essa divisão permite observar em quais cenários o modelo relacional apresentou melhor desempenho e em quais situações o modelo orientado a grafos se mostrou mais adequado.

Para permitir a conferência das execuções realizadas, os scripts, consultas e arquivos de resultado utilizados no experimento foram disponibilizados em repositório público: <https://github.com/AlisonCarv/tcc2>.

5.1 Desempenho em Consultas de Baixa Profundidade

O primeiro grupo de testes corresponde às consultas Q1, Q2 e Q3. Essas consultas avaliam cenários de recomendação com menor número de relações percorridas entre as entidades da base, como clientes, pedidos e produtos.

A Tabela 4 apresenta os tempos médios de resposta, em milissegundos, obtidos para esse grupo de consultas.

Tabela 4 - Tempo de Resposta (ms) para Consultas de Baixa Profundidade

Consulta	Regra de Negócio	PostgreSQL (Média ms)	Neo4j (Média ms)	Vantagem
Q1	Filtragem Colaborativa Simples	2,12	2,97	PostgreSQL
Q2	Coocorrência (Cesta de Compras)	0,35	2,90	PostgreSQL

Consulta	Regra de Negócio	PostgreSQL (Média ms)	Neo4j (Média ms)	Vantagem
Q3	Recomendação Híbrida (Perfil)	34,17	22,17	Neo4j

Fonte: Autoria Própria

Os resultados indicam que o PostgreSQL apresentou menor tempo médio nas consultas Q1 e Q2. Na Q1, o PostgreSQL obteve média de 2,12 ms, enquanto o Neo4j obteve média de 2,97 ms. Na Q2, voltada à análise de produtos comprados no mesmo pedido, o PostgreSQL obteve média de 0,35 ms, contra 2,90 ms do Neo4j.

Esse comportamento indica que, em consultas de menor profundidade e com relações bem definidas por chaves, o modelo relacional pode ser eficiente, especialmente quando as tabelas envolvidas possuem índices adequados. No caso da Q2, a consulta depende principalmente da comparação de itens associados ao mesmo pedido, cenário no qual a estrutura tabular do PostgreSQL apresentou vantagem.

A exceção dentro desse grupo foi a consulta Q3, na qual o Neo4j apresentou menor tempo médio: 22,17 ms contra 34,17 ms do PostgreSQL. Essa consulta combina informações de clientes da mesma cidade com categorias de produtos já consumidas, exigindo mais filtros e associações entre entidades. O resultado mostra que, mesmo em consultas de menor profundidade, o desempenho pode variar conforme a combinação de filtros, propriedades e relações envolvidas.

Assim, os resultados das consultas Q1, Q2 e Q3 não indicam superioridade absoluta de um modelo sobre o outro. Eles mostram que o PostgreSQL foi mais eficiente nos cenários mais simples de coocorrência e filtragem colaborativa, enquanto o Neo4j apresentou melhor desempenho na consulta que combinou perfil de cliente e categoria de produto.

5.2 Teste de Estresse por Aumento de Profundidade

O segundo grupo de testes corresponde à série Q4, composta pelas variações Q4_G1, Q4_G2, Q4_G3 e Q4_G4. Essas variações foram definidas para aumentar progressivamente a quantidade de relações percorridas a partir de um cliente de referência.

A Tabela 5 apresenta os tempos médios obtidos em cada grau da série Q4.

Tabela 5 - Escalonamento de Profundidade e Desempenho (ms)

Grau de Profundidade	Lógica de Vizinhaça	Qtd. de JOINS (SQL)	PostgreSQL (Média ms)	Neo4j (Média ms)
Q4_G1	Compras Diretas (1 salto)	2 JOINS	0,44	1,99
Q4_G2	Vizinhaça de 1º Nível	6 JOINS	2,78	3,28
Q4_G3	Vizinhaça de 2º Nível	10 JOINS	273,28	8,27
Q4_G4	Vizinhaça de 3º Nível	14 JOINS	[TIMEOUT]	6731,91

Fonte: Autoria própria

Na Q4_G1 e na Q4_G2, os tempos do PostgreSQL permaneceram baixos, indicando que o modelo relacional ainda conseguiu resolver as associações com custo reduzido. Entretanto, na Q4_G3, o tempo médio do PostgreSQL aumentou de 2,78 ms para 273,28 ms. Esse aumento está relacionado à maior quantidade de junções necessárias para percorrer relações sucessivas entre clientes, pedidos e produtos.

Na Q4_G4, o PostgreSQL atingiu o limite de 120 segundos definido no experimento e a execução foi registrada como timeout. Esse resultado indica que, no cenário testado, o aumento da profundidade tornou a consulta relacional custosa demais para ser concluída dentro do limite estabelecido.

O Neo4j, por sua vez, concluiu todas as variações da série Q4. Nas três primeiras variações, os tempos permaneceram baixos, passando de 1,99 ms na Q4_G1 para 8,27 ms na Q4_G3. Na Q4_G4, o tempo aumentou para aproximadamente 6,7 segundos. Esse aumento mostra que o modelo de grafos também é afetado quando a consulta percorre uma quantidade maior de relacionamentos, especialmente em regiões da base com muitos nós conectados.

Mesmo com esse aumento, o Neo4j conseguiu retornar resultado na Q4_G4, enquanto o PostgreSQL atingiu `timeout`. Portanto, no cenário avaliado, o modelo orientado a grafos apresentou melhor comportamento para consultas com maior profundidade de relações.

5.3 Descoberta de Caminhos e Expressividade

O terceiro grupo de testes corresponde à consulta Q5, cujo objetivo foi encontrar o menor caminho entre dois clientes a partir das conexões formadas por pedidos e produtos. Essa consulta foi incluída para avaliar um tipo de problema mais próximo da análise de redes, no qual a estrutura das conexões é parte central da consulta.

A Tabela 6 apresenta o tempo médio obtido na consulta Q5.

Tabela 6 - Tempo de Resposta (ms) para Descoberta de Caminho (Q5)

Consulta	Lógica Algorítmica	PostgreSQL (Média ms)	Neo4j (Média ms)	Vantagem
Q5	Shortest Path (Graus de Separação)	1595,35	9,80	Neo4j (162x mais rápido)

Fonte: Autoria própria

Na Q5, o PostgreSQL obteve média de 1595 ms, enquanto o Neo4j obteve média de 9,80 ms. A diferença observada indica vantagem significativa do Neo4j nesse cenário específico.

No PostgreSQL, a busca de caminho precisou ser implementada por meio de uma consulta recursiva, utilizando `WITH RECURSIVE`. Esse recurso permite percorrer relações sucessivas, mas exige uma formulação mais extensa e menos direta do que a consulta equivalente em grafos.

No Neo4j, a mesma lógica foi expressa por meio da função `shortestPath`, utilizada para localizar caminhos entre nós conectados (NEO4J, 2026). Como o modelo de grafos armazena os relacionamentos de forma explícita, a consulta pôde ser escrita de maneira mais próxima da estrutura do problema analisado.

Além da diferença de tempo, a Q5 evidencia uma diferença de expressividade entre SQL e Cypher. Enquanto a consulta SQL exige a construção de uma lógica recursiva, a consulta Cypher permite representar diretamente a busca de um caminho entre dois clientes. Dessa forma, no cenário de descoberta de caminhos, o Neo4j apresentou melhor desempenho e uma formulação mais simples da consulta.

6 CONSIDERAÇÕES FINAIS

Este trabalho teve como objetivo comparar o desempenho e a expressividade de consultas em dois modelos de banco de dados aplicados a um contexto de recomendação em e-commerce: o modelo relacional, implementado no PostgreSQL, e o modelo orientado a grafos, implementado no Neo4j. Para isso, foi utilizado o conjunto de dados público da Olist, modelado nos dois bancos e submetido a consultas equivalentes em SQL e Cypher.

Os resultados obtidos mostraram que não há superioridade absoluta de um modelo sobre o outro. Nas consultas de menor profundidade, especialmente em cenários de coocorrência de produtos e filtragem colaborativa simples, o PostgreSQL apresentou melhor desempenho. Esse resultado indica que, quando as relações entre os dados são pouco profundas e bem representadas por chaves e índices, o modelo relacional continua adequado e eficiente.

Por outro lado, nas consultas que exigiram maior número de relações percorridas, o Neo4j apresentou melhor comportamento. No teste de aumento progressivo de profundidade, o PostgreSQL atingiu timeout na variação mais profunda, enquanto o Neo4j concluiu a execução dentro dos limites definidos no experimento. Na consulta de menor caminho entre clientes, o Neo4j também apresentou menor tempo de resposta e uma formulação mais direta em Cypher, enquanto o PostgreSQL exigiu uma consulta recursiva em SQL.

Esses resultados indicam que a escolha entre PostgreSQL e Neo4j deve considerar o tipo de consulta predominante no sistema. Para recomendações mais simples, baseadas em relações diretas ou pouco profundas, o modelo relacional pode oferecer melhor desempenho e menor complexidade operacional. Para cenários em que a aplicação precisa percorrer redes de relações, descobrir caminhos ou analisar conexões sucessivas entre entidades, o modelo orientado a grafos tende a ser mais adequado.

Também foi possível observar diferença na expressividade das linguagens utilizadas. Em consultas tabulares e com junções diretas, SQL apresentou uma estrutura adequada ao problema. Em consultas baseadas em caminhos e relações sucessivas, Cypher permitiu representar a lógica de forma mais próxima da estrutura do grafo. Assim, a expressividade depende do tipo de problema consultado e não apenas da linguagem em si.

Como limitação, este estudo foi executado em ambiente local, com uma única máquina e com um conjunto de dados de tamanho moderado. Além disso, as consultas avaliadas representam cenários específicos de recomendação e análise de relacionamento, não abrangendo todos os tipos possíveis de aplicação em e-commerce. Portanto, os resultados devem ser interpretados dentro do escopo definido pelo experimento.

Como trabalhos futuros, sugere-se ampliar o benchmark para bases de maior volume, ambientes distribuídos ou cenários em nuvem. Também seria possível incluir outros bancos relacionais e orientados a grafos, além de avaliar estratégias híbridas, nas quais o banco relacional seja utilizado para operações transacionais e o banco de grafos para consultas de recomendação baseadas em conexões. Essas extensões podem contribuir para uma análise mais ampla sobre o uso combinado de diferentes modelos de dados em sistemas de recomendação.

REFERÊNCIAS

AGGARWAL, Charu C. **Recommender systems: the textbook**. Cham: Springer, 2016. DOI: 10.1007/978-3-319-29659-3.

CODD, Edgar F. A relational model of data for large shared data banks. **Communications of the ACM**, New York, v. 13, n. 6, p. 377-387, 1970. DOI: 10.1145/362384.362685.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison-Wesley, 2011.

JAIN, Raj. **The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation and Modeling**. New York: Wiley, 1991.

MARKETING DEAL. **Descubra quando utilizar o Graph Databases**. Deal, 14 jun. 2023. Disponível em: <https://www.deal.com.br/descubra-quando-utilizar-o-graph-databases/>. Acesso em: 4 jul. 2026.

NEO4J. **Shortest paths: Cypher Manual**. [S. l.]: Neo4j, 2026. Disponível em: <https://neo4j.com/docs/cypher-manual/current/patterns/shortest-paths/>. Acesso em: 6 jun. 2026.

OLIST. **Brazilian E-Commerce Public Dataset by Olist**. Kaggle, [s. d.]. Disponível em: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>. Acesso em: 6 jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation: index types**. [S. l.]: PostgreSQL Global Development Group, 2026a. Disponível em: <https://www.postgresql.org/docs/current/indexes-types.html>. Acesso em: 6 jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation: B-tree indexes**. [S. l.]: PostgreSQL Global Development Group, 2026b. Disponível em: <https://www.postgresql.org/docs/current/btree.html>. Acesso em: 6 jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation: genetic query optimization in PostgreSQL**. [S. l.]: PostgreSQL Global Development Group, 2026c. Disponível em: <https://www.postgresql.org/docs/current/geqo-pg-intro.html>. Acesso em: 6 jun. 2026.

RICCI, Francesco; ROKACH, Lior; SHAPIRA, Bracha; KANTOR, Paul B. (ed.). **Recommender systems handbook**. Boston: Springer, 2011. DOI: 10.1007/978-0-387-85820-3.

ROBINSON, Ian; WEBBER, Jim; EIFREM, Emil. **Graph databases: new opportunities for connected data**. 2. ed. Sebastopol: O'Reilly Media, 2015.

SADALAGE, Pramod J.; FOWLER, Martin. **NoSQL distilled: a brief guide to the emerging world of polyglot persistence**. Boston: Addison-Wesley, 2012.